

Implementation and Analysis of a Full-Stack Acoustic Network: From Sound Waves to Web Browsing

TangZhiHao 2022533131Chensian 2024531054

January 14, 2026

Abstract

This report presents the comprehensive design and implementation of a full-stack network protocol suite communicated entirely over air-gapped acoustic channels. Spanning four distinct development phases, the project constructs a robust Physical layer (PHY) using Differential Binary Phase Shift Keying (DBPSK), a reliable Link layer (MAC) employing CSMA/CA and ARQ, a Network layer (IP) capable of routing and VPN tunneling, and finally an Application layer supporting web browsing and DNS resolution. Special emphasis is placed on solving domain-specific challenges such as clock drift, multipath fading, and the strict half-duplex nature of audio hardware. The final system demonstrates the capability to route standard IPv4 traffic and perform HTTP requests over an unreliable audio link with an effective throughput of approximately 4 kbps.

1 Story

I do not much time in writing code with AI assistance, but a lot of time in solving the physical noise, and CRC Error...

2 Introduction

The objective of this series of projects was to demystify the complexities of modern network stacks by building one from scratch, using sound waves as the physical medium. Unlike traditional RF or wired links, the acoustic channel presents unique challenges: extremely low propagation speed (343 m/s), high susceptibility to environmental noise, and significant hardware latency. These constraints necessitated highly optimized designs at every layer of the OSI model. This report details the implementation of Project 1 (PHY), Project 2 (MAC), Project 3 (Network), and Project 4 (Transport/Application), culminating in a fully functional network capable of standard internet activities.

3 Project 1: The Acoustic Physical Layer (PHY)

The Physical Layer is the foundation of the system, responsible for converting digital bits into analog signals compatible with standard computer sound cards.

3.1 Task 2: Signal Generation

The first step involved interacting with the audio hardware. We utilized the `sounddevice` Python library to bypass OS audio mixing as much as possible. A composite signal $f(t) = \sin(2\pi f_1 t) + \sin(2\pi f_2 t)$ was generated to verify frequency response. This phase confirmed that the standard sampling rate of $F_s = 48$ kHz is sufficient for generating carriers up to the Nyquist limit, though we selected a conservative carrier frequency of $F_c = 6$ kHz to remain in the flattest region of the speaker/microphone frequency response curve.

3.2 Task 3: Basic Transmission System

To achieve reliable communication, we implemented a custom modem class `AudioPhysical`.

3.2.1 Modulation: DBPSK

We selected **Differential Binary Phase Shift Keying (DBPSK)** over simpler schemes like OOK (On-Off Keying) or FSK. DBPSK is resilient to amplitude variations (common as the user moves) and does not require absolute phase synchronization, only valid consecutive symbol comparison.

- **Symbol Mapping:**

$$s_k = s_{k-1} \cdot e^{j\Delta\phi_k}$$

where $\Delta\phi_k = 0$ for bit '0' and $\Delta\phi_k = \pi$ for bit '1'.

- **Pulse Shaping:** Rectangular pulses were used for simplicity, with a Symbol Rate $R_s = 4000$ baud. This yields $N_{sps} = 48000/4000 = 12$ samples per symbol.

3.2.2 Frame Structure

To define packet boundaries in a continuous stream of audio samples, a rigorous frame structure was designed:

```
+-----+-----+-----+-----+
| Preamble (13) | Length (16 bits) | Payload   | CRC (32) |
+-----+-----+-----+-----+
```

Synchronization: We utilized a 13-bit Barker-like sequence (1111100110101) as the preamble. The receiver maintains a sliding window of the incoming signal, computing the cross-correlation with the reference preamble $p[n]$:

$$(f \star g)[n] = \sum_{m=-\infty}^{\infty} \overline{f[m]} g[m+n]$$

A valid frame is detected when the correlation peak magnitude exceeds a dynamic noise floor threshold (typically $> 10 \times$ noise floor).

Listing 1: Preamble Detection Code

```
# Cross Correlate magnitude
corr = signal.correlate(baseband, expected_preamble, mode='valid')
peak_val = np.max(np.abs(corr))

if peak_val > self.noise_floor * 10:
    # Packet Start Detected
    start_idx = np.argmax(np.abs(corr))
```

3.2.3 Adaptive Clock Recovery (Crucial Optimization)

A major challenge discovered during testing was **Sampling Rate Offset (SRO)**. Two sound cards set to 48 kHz rarely operate at exactly the same rate; a separate crystal oscillator might run at 48,004 Hz vs 47,995 Hz. Over a 1000-byte packet, this drift accumulates, causing the receiver to sample the wrong part of the symbol (Inter-Symbol Interference).

Solution: We implemented a grid-search algorithm during the preamble processing. The receiver attempts to decode the length header using a range of "drift" factors (e.g., ± 2000 ppm). The drift factor δ that minimizes the imaginary component of the differential decoding (pure BPSK should be real) is locked in:

$$\text{Cost}(\delta) = \sum_k |\text{Im}(r_k \cdot r_{k-1}^*)|$$

3.3 Advanced PHY Features (Optional Tasks)

- **Task 4: Error Correction (FEC):** We integrated a Reed-Solomon codec (`reedsolo`). For testing where libraries were unavailable, a fallback Repetition Code (3x redundancy with majority vote) was implemented.

- **Task 5: OFDM:** A basic Orthogonal Frequency Division Multiplexing PHY was implemented using `numpy.fft`. The 6 kHz bandwidth was divided into 64 subcarriers, utilizing QPSK mapping and a Cyclic Prefix (CP) to combat multipath delay spread.
- **Task 6: Chip Dream (Hardware Optimization):** To simulate hardware constraints (e.g., FPGA implementation), we implemented a fixed-point arithmetic module.
 - **Lookup Tables (LUT):** Instead of calculating `sin()`/`cos()` at runtime, we pre-calculated a 1024-entry table mapping phase angles $[0, 2\pi)$ to `int16` values.
 - **Q1.15 Arithmetic:** All signal processing operations were converted to integer math using Q1.15 fixed-point format (1 sign bit, 15 fractional bits).
- **Task 8: Range Challenge:** By combining the robust DBPSK modulation (Task 3) with the Reed-Solomon FEC (Task 4) and increasing the calibrated noise threshold detection, we successfully achieved reliable transmission at distances exceeding **2 meters** in a noisy lab environment.

4 Project 2: The Link Layer (MAC)

With a working PHY, Project 2 focused on creating a regulated channel where multiple nodes could communicate without constant collisions.

4.1 Task 1: Framing & Addressing

The MAC layer wraps the raw payload with addressing information. The 3-byte header consists of [Dest ID] [Src ID] [Type]. This allows the MAC software to filter packets. Protocol types (DATA, ACK, NACK) distinguish the nature of the traffic.

4.2 Task 2: Stop-and-Wait ARQ

The acoustic channel involves significant latency (processing + propagation). A simple Stop-and-Wait protocol was deemed most robust.

- **Sender:** Transmits a chunk and starts a timer $T_{ack} = 5.0s$.
- **Receiver:** Upon validating the CRC, waits `MAC_SIFS` (0.2s) and sends an ACK.
- **Retransmission:** If the timer expires, the sender retries up to `MAX_RETRIES` (5 times) before declaring link failure.

4.3 Task 3: CSMA/CA

To support the multi-node topology, ****Carrier Sense Multiple Access with Collision Avoidance**** was implemented.

1. **Carrier Sense:** Before sending, the node calculates the RMS energy of the last N audio samples. If $E_{rms} > \text{Threshold}_{cs}$, the channel is busy.
2. **Backoff Strategy:** If busy, the node enters a backoff state. The wait time is calculated as:

$$T_{wait} = \text{Uniform}(0, 2^{BE} - 1) \times T_{slot}$$

where BE (Backoff Exponent) is incremented after each busy sensing ($BE = \min(BE+1, BE_{max})$), implementing Binary Exponential Backoff.

5 Project 3: The Network Layer (IP)

Project 3 bridged the gap between our custom acoustic protocol and the standard Internet Protocol stack.

5.1 Task 1: Integration with Scapy

We utilized `scapy` to handle IP packet construction. The `NetworkLayer` parses valid MAC frames as `IP()`. If `pkt[IP].dst == self.ip`, it is processed locally (e.g., ICMP Echo Reply).

5.2 Task 2 & 4: Router with NAT and Traversal

The `Router` class (and `NatRouter` subclass) implements full Masquerading (NAPT).

- **SNAT (Source NAT):** For outgoing traffic (Audio $\rightarrow$ Internet), the router maps (`SrcIP`, `SrcPort`) to (`RouterPublicIP`, `NewAllocatedPort`).
- **DNAT (Destination NAT):** For incoming traffic (Internet $\rightarrow$ Audio), the router uses the table to map (`RouterPublicIP`, `NewAllocatedPort`) back to the internal (`SrcIP`, `SrcPort`).

Listing 2: NAT Logic Snippet

```
if key in self.reverse_nat_table:
    ext_port = self.reverse_nat_table[key]
else:
    ext_port = self._get_free_port()
    self.nat_table[(proto, ext_port)] = (pkt.src, sport)

# Rewrite Source to Router's Public IP
pkt.src = self.public_ip
if proto == 'TCP': pkt[TCP].sport = ext_port
```

5.3 Task 5: ARP (Address Resolution Protocol)

A dynamic ARP mechanism was implemented. When `send_ip_packet` encounters an unknown destination IP, the node broadcasts an ARP Request ('Who has X_i '). The target replies with its MAC ID.

```
[ARP] Who has 192.168.10.2? (Broadcast)
[ARP] 192.168.10.2 is at Node 2 (Unicast Reply)
```

6 Project 4: Applications

The final project focused on high-level applications, proving the stack's utility.

6.1 Task 1: User-Space TCP

Since the OS TCP stack is too aggressive for our 4 kbps link, we implemented a userspace TCP socket in `src/tcp.py`.

- **State Machine:** Implements `SYN_SENT`, `ESTABLISHED`, `FIN_WAIT` states.
- **Handshake:** Robust 3-way handshake with extended timeouts (8s) to survive acoustic latency.
- **Flow Control:** Stop-and-wait ARQ for data segments. MSS is clamped to 300 bytes to prevent fragmentation.

6.2 Task 3: DNS Server

To enable domain name resolution for the optional web browsing task, we implemented a custom DNS Server (`src/dns_server.py`).

- ****Recursive Resolution:**** The server listens on UDP port 53. When it receives a standard DNS Query (A Record), it acts as a proxy, observing the query and resolving it using the host OS's internet connection (`socket.gethostbyname`).

- ****Response Generation:**** It constructs a valid DNS Response packet using Scapy, setting the **QR=1** (Response) flag and appending the **DNSRR** (Resource Record) with the resolved IP.

Listing 3: DNS Handler

```
def handle_dns_query(self, pkt):
    dns = pkt[DNS]
    qname = dns.qd[0].qname.decode('utf-8')
    resolved_ip = socket.gethostbyname(qname.rstrip('.'))

    # Construct Response
    resp = IP(dst=pkt[IP].src) / UDP(dport=pkt[UDP].sport) / \
        DNS(id=dns.id, qr=1, aa=0,
            qd=dns.qd,
            an=DNSRR(rrname=dns.qd[0].qname, rdata=resolved_ip))
    self.net.send_ip_packet(pkt[IP].src, resp)
```

6.3 Task 4: The "Curl" Proxy (HTTP)

To browse the web over the low-bandwidth audio link, a custom UDP-based protocol was managed.

- ****Protocol:**** `META|<URL> → META|<Size>|<Chunks> → CHUNK|<ID> → DATA|<ID>|<Bytes>`.
- This "Application-Layer ARQ" allows fetching large HTML files chunk-by-chunk without TCP overhead.

6.4 Optional: Real Web Browsing via VPN

Leveraging the VPN (TUN/TAP) from Project 3 and the DNS Server (Task 3), we routed Google Chrome traffic through the acoustic link. The browser sends a standard DNS query for `example.com`, which our stack intercepts and resolves. The subsequent HTTP GET is routed via TUN, encapsulated, transmitted over audio, and NAT-ed to the real internet.

7 Challenges and Solutions

7.1 The Half-Duplex Constraint

Standard consumer audio hardware, unlike specialized RF equipment, struggles to perform full-duplex communication on the same frequency band. When a device transmits, the microphone captures its own high-volume output ("Self-Interference"), drowning out any incoming signals from remote nodes. **Solution:** We enforced a strict Half-Duplex state machine in the MAC layer.

- **TX/RX Mutual Exclusion:** A mutex lock ensures that `transmit()` cannot be called while `receive_loop` is actively processing a frame candidate.
- **Echo Cancellation (Naive):** Immediately after a transmission finishes, the receiver buffer is flushed (`queue.clear()`). This discards the high-energy samples of our own transmission that were recorded by the input stream, preventing the PHY from falsely detecting its own packet as a collision or a new incoming frame.

7.2 Environmental Noise and Thresholding

Initial tests in a lab environment failed due to intermittent background noise (e.g., HVAC systems, talking). A static threshold for Carrier Sense proved unreliable: setting it too high caused deafness to weak signals, while setting it too low caused constant false positives (Carrier Busy). **Solution:** The `calibrate_noise()` function was introduced. On system startup, the node enters a "Learning Mode" for 2 seconds, sampling the ambient spectral energy density.

$$\text{Threshold}_{cs} = \alpha \times \mu_{noise} \quad (\text{where } \alpha \approx 10)$$

This dynamic threshold adaptively effectively creates a "Squelch" control, allowing the system to operate in libraries (quiet) or hallways (noisy) without code recompilation.

7.3 Multipath Propagation and Inter-Symbol Interference (ISI)

In a reflective environment like a classroom, the acoustic signal bounces off walls and desks, arriving at the receiver via multiple paths with varying delays. This delay spread (τ_d) causes symbols to bleed into neighboring symbol periods (ISI). **Solution:** Our choice of a relatively low Symbol Rate ($R_s = 4000$ baud) compared to the carrier (6 kHz) provides a symbol period $T_s = 250\mu s$. While this limits throughput, it ensures that the reflections from nearby objects (delay < 1 meter ≈ 3 ms) are effectively integrated into the symbol energy or discarded by the preamble correlator's sharpness. For Project 1 Task 5 (OFDM), the Cyclic Prefix (CP) was specifically tuned to exceed this maximum delay spread.

8 System Analysis & Future Work

8.1 Performance Bottlenecks

The current system achieves an effective throughput of ≈ 4 kbps. Analysis reveals the primary bottlenecks:

1. **Protocol Overhead:** The framing structure imposes significant overhead. A 100-byte payload requires 13 bits (Preamble) + 16 bits (Len) + 32 bits (CRC) + 3 bytes (MAC Header) + 20 bytes (IP Header). For small packets (like ACK), the efficiency drops below 50%.
2. **Stop-and-Wait ARQ:** The "Ping-Pong" nature of the MAC layer leaves the channel idle for the duration of the propagation delay + processing time ($T_{prop} + T_{proc}$) between every frame. A Sliding Window protocol would significantly improve utilization.

8.2 Sugestions for Future Improvements

- **QPSK Modulation:** Currently, we use BPSK (1 bit/symbol). Upgrading to Quadrature Phase Shift Keying (QPSK) would double the raw data rate to 8 kbps without increasing bandwidth usage, provided the Signal-to-Noise Ratio (SNR) is sufficient.
- **Forward Error Correction (FEC):** While Reed-Solomon was implemented, a convolutional code (e.g., Viterbi algorithm) might offer better performance for the burst errors common in acoustic channels due to transient noise (e.g., a dropped pen).
- **Hardware Acceleration:** The Python implementation suffers from garbage collection pauses. Porting the PHY layer to C or using `numba` JIT compilation would reduce processing latency, allowing for tighter ACK timeouts and faster collisions detection.

9 Conclusion

This project successfully demonstrated the viability of "Internet over Sound". By rigorously applying the layered network architecture, we built a system where a high-level application (like a web browser) can function unaware that its packets are traveling as chirps and beeps through the air. While the throughput (≈ 4 kbps) is modest by modern standards, the system's robustness against clock drift, noise, and collisions proves the efficacy of the implemented protocols.

10 Suggestion

Please provide a more reliable physical hardware next time, Or please give a robust physical implementation for us, to let us use it for higher layer. We are attending the Computer Network Course, Not the AUDIO SIGNAL PROCESSING!

A typical Working Environment:

